

Grouping Data

In this chapter, you'll learn how to group data so you can summarize subsets of table contents. Grouping data involves two `SELECT` statement clauses that you haven't learned about yet: the `GROUP BY` clause and the `HAVING` clause.

Understanding Data Grouping

In the previous chapter, you learned that the SQL aggregate functions can be used to summarize data. By using those functions, you can count rows, calculate sums and averages, and obtain high and low values without having to retrieve all the data.

All the calculations thus far have been performed on all the data in a table or on data that matched a specific `WHERE` clause. As a reminder, the following example returns the number of products offered by vendor 1003:

► Input

```
SELECT Count(*) AS num_prods
FROM products
WHERE vend_id = 1003;
```

► Output

num_prods
7

But what if you want to return the number of products offered by each vendor? Or products offered by vendors who offer a single product? Or products offered by vendors who offer more than 10 products?

This is where groups come into play. Grouping enables you to divide data into logical sets so you can perform aggregate calculations on each group.

Creating Groups

You create groups in MySQL by using the `GROUP BY` clause in a `SELECT` statement. The best way to understand this is to look at an example:

► Input

```
SELECT vend_id, Count(*) AS num_prods
FROM products
GROUP BY vend_id;
```

► Output

vend_id	num_prods
1001	3
1002	2
1003	7
1005	2

► Analysis

This `SELECT` statement specifies two columns: `vend_id`, which contains the ID of a product's vendor, and `num_prods`, which is a calculated field (created using the `Count(*)` function). The `GROUP BY` clause instructs MySQL to sort the data and group it by `vend_id`. This causes `num_prods` to be calculated once per `vend_id` rather than once for the entire table. As you can see in the output, vendor 1001 has 3 products listed, vendor 1002 has 2 products listed, vendor 1003 has 7 products listed, and vendor 1005 has 2 products listed.

By using `GROUP BY`, you do not have to specify each group to be evaluated and calculated. That happens automatically. The `GROUP BY` clause instructs MySQL to group the data and then perform the aggregate calculation on each group rather than on the entire result set.

Before you use `GROUP BY`, here are some important rules about its use that you need to know:

- A `GROUP BY` clause can contain as many columns as you want. You can nest groups, which means you have more granular control over how data is grouped.
- If you have nested groups in your `GROUP BY` clause, data is summarized at the last specified group. In other words, all the columns specified are evaluated together when grouping is established (so you won't get data back for each individual column level).
- Every column listed in `GROUP BY` must be a retrieved column or a valid expression (but not an aggregate function). If an expression is used in the `SELECT`, that same expression must be specified in `GROUP BY`. You cannot use aliases.
- Aside from the aggregate calculations statements, every column in your `SELECT` statement should be present in the `GROUP BY` clause.

- If the grouping column contains a row with a `NULL` value, `NULL` will be returned as a group. If there are multiple rows with `NULL` values, they'll all be grouped together.
- The `GROUP BY` clause must come after any `WHERE` clause and before any `ORDER BY` clause.

Tip

Using ROLLUP To obtain values for each group and at a summary level (for each group), use the `WITH ROLLUP` keyword, as shown here:

```
SELECT vend_id, COUNT(*) AS num_prods
FROM products
GROUP BY vend_id WITH ROLLUP;
```

Filtering Groups

In addition to allowing you to group data by using `GROUP BY`, MySQL also allows you to filter which groups to include and which to exclude. For example, you might want a list of all customers who have made at least two orders. To obtain this data, you must filter based on the complete group, not on individual rows.

You've already seen the `WHERE` clause in action (introduced in Chapter 6, "Filtering Data"). But `WHERE` does not work for grouping because `WHERE` filters specific rows, not groups. As a matter of fact, `WHERE` has no idea what a group is.

So what do you use instead of `WHERE`? MySQL provides yet another clause for this purpose: the `HAVING` clause. `HAVING` is very similar to `WHERE`. In fact, all types of `WHERE` clauses you've learned about thus far can also be used with `HAVING`. The only difference is that `WHERE` filters rows, and `HAVING` filters groups.

Tip

HAVING Supports All of WHERE's Operators In Chapter 6 and Chapter 7, "Advanced Data Filtering," you learned about `WHERE` clause conditions (including wildcard conditions and clauses with multiple operators). All the techniques and options you learned about for `WHERE` can also be applied to `HAVING`. The syntax is identical; just the keyword is different.

So how do you filter rows? Look at the following example:

► Input

```
SELECT cust_id, Count(*) AS num_orders
FROM orders
GROUP BY cust_id
HAVING Count(*) >= 2;
```

► Input

cust_id	num_orders
10001	2

► Analysis

The first three lines of this `SELECT` statement are similar to the statements shown previously. The final line adds a `HAVING` clause that filters on those groups with `Count(*)` `>= 2` (that is, two or more orders). As you can see, a `WHERE` clause does not work here because the filtering is based on the group aggregate value, not on the values of specific rows.

Note

The Difference Between `HAVING` and `WHERE` Here's another way to look at it: `WHERE` filters before data is grouped, and `HAVING` filters after data is grouped. This is an important distinction; rows that are eliminated by a `WHERE` clause are not included in the group. Eliminating rows could change the calculated values, which in turn could affect which groups are filtered based on the use of those values in the `HAVING` clause.

So is there ever a need to use both `WHERE` and `HAVING` clauses in one statement? Actually, yes, there is. Say that you need a list of all vendors who have 2 or more products priced at 10 or more. To do this, you can add a `WHERE` clause that filters out products that cost less than 10. You then add a `HAVING` clause to filter just the groups with two or more rows in them.

Here's an example:

► Input

```
SELECT vend_id, Count(*) AS num_prods
FROM products
WHERE prod_price >= 10
GROUP BY vend_id
HAVING Count(*) >= 2;
```

► Output

vend_id	num_prods
1003	4
1005	2

► Analysis

This statement warrants an explanation. The first line is a basic `SELECT` that uses an aggregate function—much like the examples thus far. The `WHERE` clause filters all rows with `prod_price` of at least 10. Data is then grouped by `vend_id`, and then a `HAVING` clause filters just those groups with a count of 2 or more. Without the `WHERE` clause, two extra rows would have been retrieved (vendor 1002, who only sells products all priced under 10, and vendor 1001, who sells 3 products but only one of them is priced greater or equal to 10), as shown here:

► Input

```
SELECT vend_id, Count(*) AS num_prods
FROM products
GROUP BY vend_id
HAVING Count(*) >= 2;
```

► Output

```
+-----+-----+
| vend_id | num_prods |
+-----+-----+
| 1001 | 3 |
| 1002 | 2 |
| 1003 | 7 |
| 1005 | 2 |
+-----+-----+
```

Grouping and Sorting

It is important to understand that `GROUP BY` and `ORDER BY` are very different, even though they often accomplish the same thing. Table 13.1 summarizes the differences between them.

Table 13.1 ORDER BY Versus GROUP BY

ORDER BY	GROUP BY
Sorts generated output.	Groups rows. The output might not be in group order, however.
Any columns (even columns not selected) may be used.	Only selected columns or expressions columns may be used, and every selected column must be used.
Never required.	Required if using columns (or expressions) with aggregate functions.

The first difference listed in Table 13.1 is extremely important. More often than not, you will find that data grouped using `GROUP BY` will indeed be output in group order. But that is not always the case, and it is not actually required by the SQL specifications. Furthermore, you might actually want it sorted differently than it is grouped.

Just because you group data one way (to obtain group-specific aggregate values) does not mean that you want the output sorted that same way. You should always provide an explicit `ORDER BY` clause as well, even if it is identical to the `GROUP BY` clause.

Tip

Don't Forget `ORDER BY` As a rule, any time you use a `GROUP BY` clause, you should also specify an `ORDER BY` clause. That is the only way to ensure that data is sorted properly. Never rely on `GROUP BY` to sort your data.

To demonstrate the use of both `GROUP BY` and `ORDER BY`, let's look at an example. The following `SELECT` statement is similar to the ones shown previously. It retrieves the order number and total order price of all orders with a total price of 50 or more:

► Input

```
SELECT order_num,
       Sum(quantity*item_price) AS ordertotal
FROM orderitems
GROUP BY order_num
HAVING Sum(quantity*item_price) >= 50;
```

► Output

order_num	ordertotal
20005	149.87
20006	55.00
20007	1000.00
20008	125.00

To sort the output by order total, all you need to do is add an `ORDER BY` clause, as follows:

► Input

```
SELECT order_num,
       Sum(quantity*item_price) AS ordertotal
FROM orderitems
GROUP BY order_num
HAVING Sum(quantity*item_price) >= 50
ORDER BY ordertotal;
```

► Output

order_num	ordertotal
20006	55.00

	20008	125.00	
	20005	149.87	
	20007	1000.00	
+-----+-----+			

► Analysis

In this example, the `GROUP BY` clause is used to group the data by order number (the `order_num` column) so that the `Sum(*)` function can return the total order price. The `HAVING` clause filters the data so that only orders with a total price of 50 or more are returned. Finally, the output is sorted using the `ORDER BY` clause.

Combining Grouping and Data Summarization

In Chapter 12, “Summarizing Data,” you learned how to use functions like `Count()`, `Min()`, `Sum()`, and so on. You can also use these functions when grouping data to perform sophisticated reporting.

For example, the `orders` table contains a list of all orders placed. Each order has an order date in a column (appropriately named) `order_date`. What if you needed to know the best sales month? Determining this requires counting sales per month, which in turn requires extracting the year and month from `order_date`.

Here’s the code:

► Input

```
SELECT Year(order_date) AS order_year,
       Month(order_date) AS order_month,
       Count(*) AS orders_placed
FROM orders
GROUP BY order_year, order_month
ORDER BY orders_placed DESC
```

► Output

+-----+-----+-----+			
	order_year	order_month	orders_placed
+-----+-----+-----+			
	2023	9	3
	2023	10	2
+-----+-----+-----+			

► Analysis

The `Year()` and `Month()` functions extract the year and month, respectively, from `order_date`, and the values are assigned to aliases. `GROUP BY` uses those aliases to group the retrieved data and returns one row per month per year. As data is grouped by year and month, `Count(*)` counts the number of rows for each month, which is exactly what we need. The output is sorted by `ORDER BY orders_placed DESC` so that the months are listed in descending order of sales.

SELECT Clause Ordering

This is probably a good time to review the order in which `SELECT` statement clauses are to be specified. Table 13.2 lists all the clauses you have learned thus far, in the order they must be used.

Table 13.2 SELECT Clauses and Their Sequence

Clause	Description	Required
SELECT	Columns or expressions to be returned	Yes
FROM	Table to retrieve data from	Only if selecting data from a table
WHERE	Row-level filtering	No
GROUP BY	Group specification	Only if calculating aggregates by group
HAVING	Group-level filtering	No
ORDER BY	Output sort order	No
LIMIT	Number of rows to retrieve	No

Summary

In Chapter 12, you learned how to use the SQL aggregate functions to perform summary calculations on your data. In this chapter, you learned how to use the `GROUP BY` clause to perform calculations on groups of data and return results for each group. You saw how to use the `HAVING` clause to filter specific groups. You also learned the difference between `ORDER BY` and `GROUP BY` and between `WHERE` and `HAVING`.

Challenges

1. The `OrderItems` table contains the individual items for each order. Write a SQL statement that returns the number of lines (as `order_lines`) for each order number (`order_num`) and sort the results by `order_lines`.
2. Write a SQL statement that returns a field named `cheapest_item`, which contains the lowest-cost item for each vendor (using `prod_price` in the `Products` table) and sort the results from lowest to highest cost.
3. It's important to identify the best customers. Write a SQL statement to return the order number (`order_num` in the `OrderItems` table) for every order of at least 100 items.

4. Another way to determine the best customers is based on how much they have spent. Write a SQL statement to return the order number (`order_num` in the `OrderItems` table) for every order with a total price of at least 1000. Here's a hint: For this one, you'll need to calculate and sum the total (`item_price` multiplied by `quantity`). Sort the results by order number.
5. What is wrong with the following SQL statement? (Try to figure it out without running it.)

```
SELECT order_num, COUNT(*) AS items
FROM OrderItems
GROUP BY items
HAVING COUNT(*) >= 3
ORDER BY items, order_num;
```

This page intentionally left blank